

VisionGuard **ULTRA**

“Every camera becomes a guard that never gets tired.”

AI that finds emergencies in CCTV video and sends help — automatically.

Abdul Rafay · Moiz · Areeba · Aqib

Cameras record. Nobody watches.

“The camera sees the crime — but nobody acts.”

20M+

Too many screens

One operator cannot watch 50 cameras at the same time.

~20 min

Human fatigue

Attention drops after about 20 minutes of watching screens.

After

Late help

CCTV gives proof after the crime — not help during it.

What we want to achieve

“Find danger fast, decide smart, send the right help.”

- 1 **Watch every camera** at the same time — no tired humans
- 2 **Find 12 types** of emergency in real time
- 3 **Few false alarms** — target under 3–5 wrong alerts in 100
- 4 **Send to the right department** with sealed proof
- 5 **Protect privacy** — no face recognition

VisionGuard in one line

“Detect → Decide → Dispatch.”

Detect

AI models look at every frame and every sound.

Decide

Rules check the danger over time and give a risk score 0–100.

Dispatch

Alert goes to Police 15, Fire 16, Edhi 115 or Rescue 1122 with video proof.

Works on cameras the city already has — no new cameras needed.

How we built it — 6 steps

“Start simple, measure, improve.”

1. Study the problem

Karachi threats, operator fatigue, helplines, privacy.

2. Build a baseline

Ready YOLOv8 + tracking + simple colour fire rule.

3. Prepare data

Collect, clean, augment and balance datasets.

4. Train our models

Fine-tune YOLOv8 for fire, weapon and violence.

5. Add rules & actions

12 event rules, risk score, routing, evidence.

6. Test & improve

Measure accuracy and speed, fix, repeat.

Six phases from camera to action

“Each phase has one job — and a clear reason.”

1

Input

Camera stream

RTSP · webcam · file

Why threaded capture: Each camera runs on its own thread with auto-reconnect and keeps only the newest frame, so delay never builds up.

2

Detection

YOLOv8s + ByteTrack

480 px · one pass

Why YOLOv8s: One-stage, anchor-free and fast on a 4 GB GPU. Detection and tracking share one call — about 2x faster.

3

Specialists

Pose · Fire ·
Weapon · Violence

async worker

Why a second thread: Heavy models run in the background on the newest frame, so the live video never freezes (+43% FPS).

4

Event Engine

12 temporal rules

1.5 s persistence

Why time rules: One frame can be wrong. A danger must last 1.5 s (weapon 0.5 s); a grace period ignores flicker.

5

Risk & Route

Score 0–100

85+ = critical

Why a risk score: Operators see the most dangerous event first; each event type goes to the right helpline.

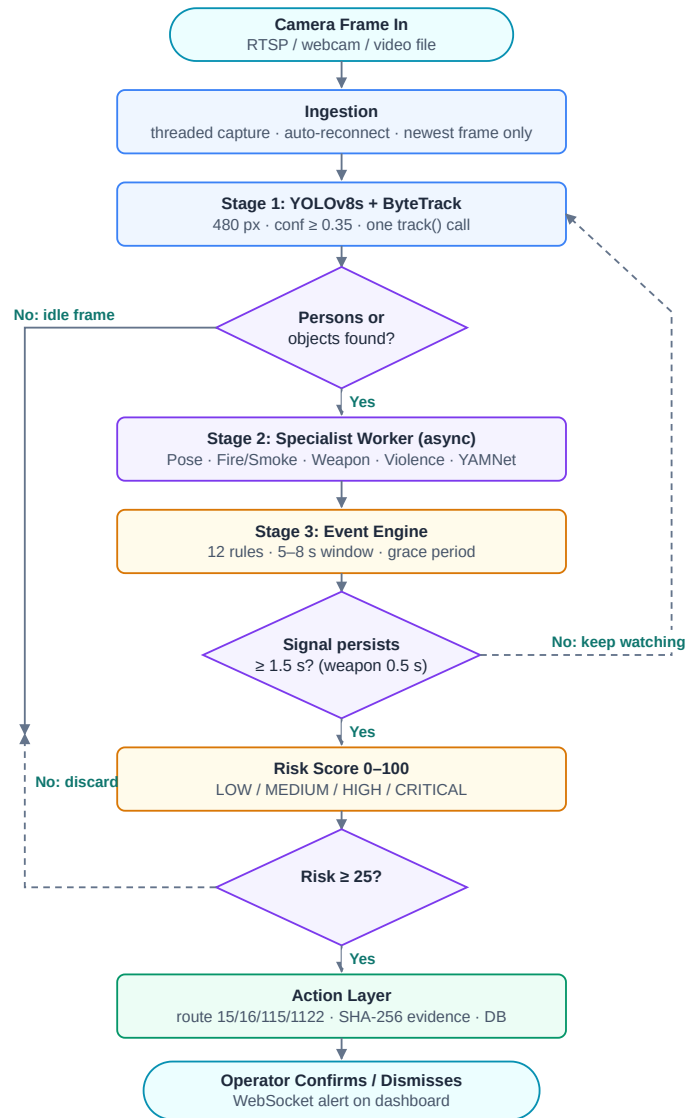
6

Evidence & UI

SHA-256 · WebSocket

5 s + 3 s clip

Why sealed evidence: Any edit changes the fingerprint, so clips are trustworthy; alerts reach the dashboard instantly.



6 · LOGICAL FLOW

Decision gates

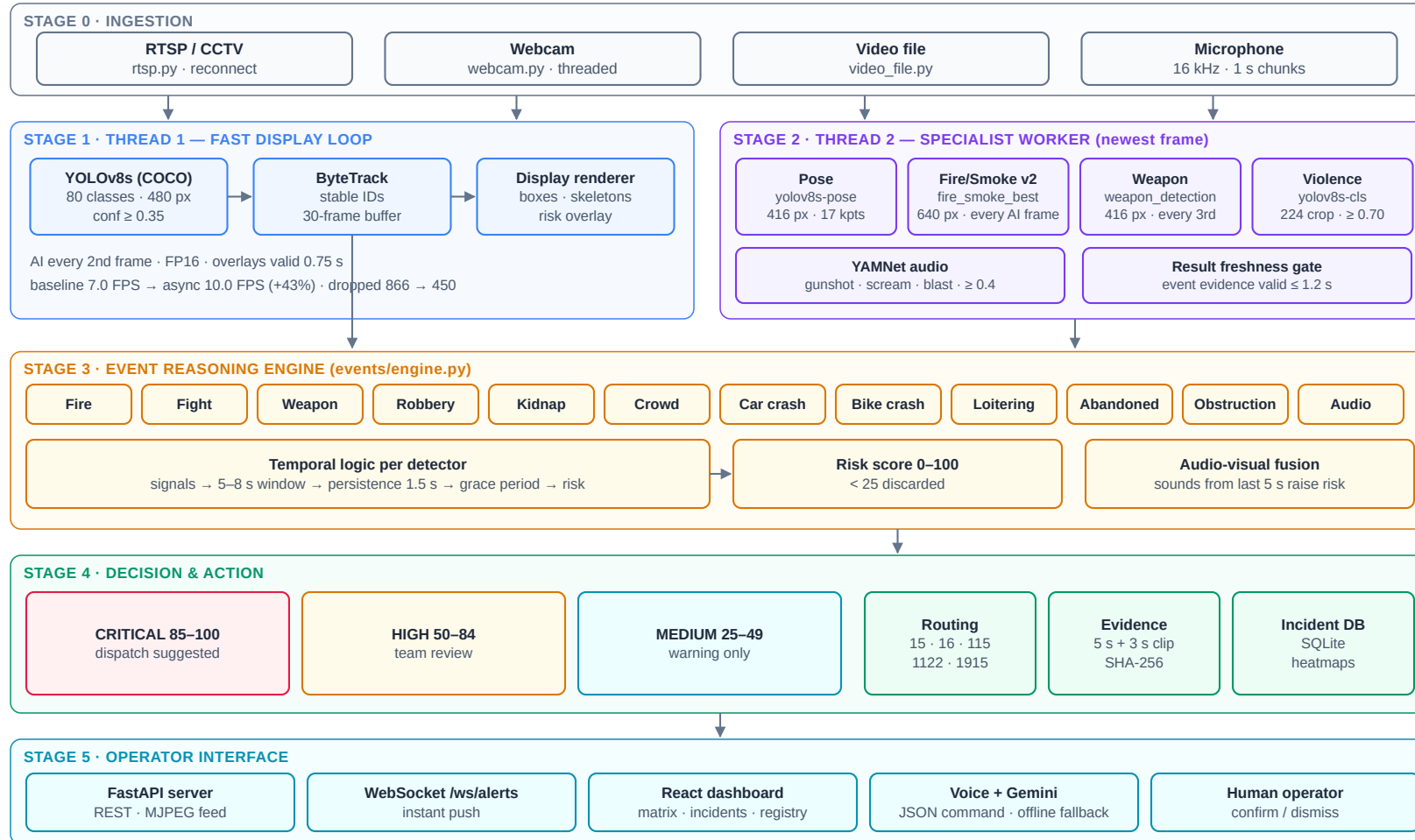
“The system asks three questions before it raises an alarm.”

- 1 **Gate 1 — Anything there?** If YOLO finds no people or objects, the frame is skipped.
- 2 **Gate 2 — Does it last?** The danger must continue 1.5 s (weapon 0.5 s). Otherwise we keep watching.
- 3 **Gate 3 — Is it serious?** Risk below 25 is discarded; 85+ is critical.
- 4 **Human gate:** the operator confirms or dismisses every dispatch.

7 · SYSTEM ARCHITECTURE

Complete end-to-end pipeline

“Two lanes: one keeps video smooth, one does the heavy thinking.”



Rules that think over time

“One frame can lie — a few seconds tell the truth.”

- 1 Collect **signals** from every frame
- 2 Keep a **time window** of the last 5–8 seconds
- 3 Event must last **1.5 s** (weapon: **0.5 s**)
- 4 **Grace period**: one missed frame does not reset
- 5 Give a **risk score** 0–100

SCORE	LEVEL	ACTION
85–100	CRITICAL	Send help
50–84	HIGH	Team review
25–49	MEDIUM	Warning
0–24	LOW	Ignore

Example — Fight = 2+ people close + fast wrists + repeats 1.5 s + violence model $\geq 70\%$.

The AI brains inside

“Ready-made eyes, plus 3 specialists we trained ourselves.”

MODEL	JOB	TYPE	MADE BY
YOLOv8s	Find people, cars, bags (80 objects)	Detection	Pre-trained (COCO)
YOLOv8s-pose	17 body points per person	Pose	Pre-trained
Fire & Smoke v2	Find fire and smoke	Detection	Trained by us
Weapon	Find knife, pistol, rifle	Detection	Trained by us
Violence	Is this person fighting? yes / no	Classification	Trained by us
YAMNet	Gunshot, scream, explosion	Audio	Pre-trained
ByteTrack	Keep the same ID for each person	Tracking	Algorithm

Why YOLOv8s? Fast on a small 4 GB GPU, one tool for boxes, body points and yes/no.

What our models learned from

“Good data makes a good model.”

16,027

Fire & Smoke

D-Fire + Roboflow fire/smoke

7,116

Weapon

CCTV weapon, knife, pistol
sets

~12,000

Violence

RWF-2000 + Real Life
Violence + CCTV

4,304

Weather extra

made by our augmentation

Total for our models

~35,100 real images + 4,304 augmented
≈ 39,400 images

Boxes checked

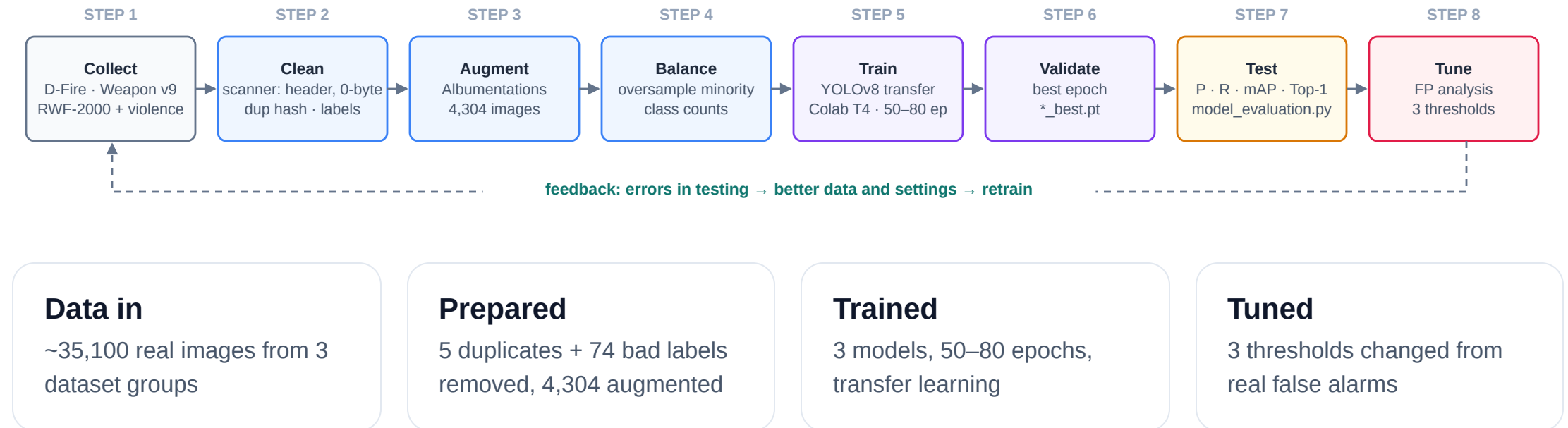
32,836 labels (5,187 fire/smoke + 27,649
weapon)

Pre-trained on

COCO 118,000 images · AudioSet sound
clips

From raw data to a trained model

“Clean it, grow it, balance it — then teach it.”



Clean → Augment → Balance

“Real CCTV is messy — so we trained on messy data.”

1

Clean

Checked 23,000+ images. Removed 5 copied images and 74 broken label lines.
0 broken images in D-Fire.

2

Augment

Added rain, fog, shadow, light change, rotation and camera noise → 4,304 new images.

3

Balance

Weapon classes were uneven (11,545 vs 7,532 boxes). Small classes shown more often so the model is not biased.

How we taught the models

“Don't start from zero — teach a model that already knows how to see.”

MODEL	INPUT	EPOCHS
Fire & Smoke	640 × 640	80
Weapon	640 × 640	60
Violence	224 × 224	50

Google Colab · free T4 GPU · about 3–6 hours each.

- 1 **Transfer learning:** start from YOLOv8 that already knows shapes
- 2 **Epoch** = model sees all images once
- 3 Fire gets most epochs — **no fixed shape**, hardest
- 4 Too many epochs = **memorising**, so we keep the best version

How we measure accuracy

“Precision = how often an alarm is right. Recall = how many dangers we catch.”

WORD	SIMPLE MEANING
Precision	When it says “fire”, how often it is right
Recall	Of all real fires, how many it found
mAP@50	Overall box quality (box overlaps $\geq 50\%$)
mAP@50-95	Same, but much stricter
Top-1	Classifier's first answer is correct

Our pass marks

Detection mAP@50 $\geq 70\%$ · Classification Top-1 $\geq 85\%$ · False alarms $< 5\%$ on 20 test videos.

Our test tools

model_evaluation.py · false_positive_analyzer.py · speed benchmarks

Fire & Smoke model — measured

“Twice as good as our old method — and we know how to make it better.”

47.6%

Precision

old colour method: ~25%

25.5%

Recall

finds 1 in 4 boxes per frame

16.0%

mAP@50

old: ~8% → 2× better

7.6%

mAP@50-95

old: ~3%

Why numbers look low

D-Fire is hard: tiny, far, hidden fires and soft smoke. But alerts use time — a fire missed in one frame is caught in the next.

Fire vs smoke

Fire precision 56.1% · Smoke 39.1%. Weapon & violence tests: in progress.

What we made better

“Measure → find the problem → fix → measure again.”

WHAT	BEFORE	AFTER	GAIN
Fire model	Colour rule	Trained AI	2× mAP, +90% precision
Pipeline	One slow thread	Two lanes (async)	+43% FPS (7 → 10)
Dropped frames	866	450	−48%
Crowd alarm	0.90	0.95	fewer false crowds
Time before alert	1.0 s	1.5 s	no duplicate alerts
Confidence	0.30	0.35	fewer dark-video errors

Safe, fair and private

“We watch actions — not faces.”

No face ID

No face recognition, no face database.

Human decides

AI suggests, operator confirms.

Sealed proof

5 s before + 3 s after, SHA-256 fingerprint.

Login only

Only allowed users see data.

Honest status

“We know where we are — and where we are going.”

GOAL	TODAY	
12 emergency types	12 rules working	● Done
Sealed evidence + privacy	Working	● Done
Speed 25–30 FPS	10 FPS on a 4 GB GPU	● In progress
Detection mAP@50 $\geq$ 70%	Fire 16%	● Not yet
Violence / weapon tests	Running	● In progress
False alarms < 5%	Measuring	● In progress

Where we go from here

“From a strong prototype to a real city system.”

- 1 **Better models:** more epochs, bigger YOLO, local Karachi data
- 2 **Finish testing:** weapon & violence reports, 20-video false-alarm test
- 3 **Night vision:** low-light training and IR cameras
- 4 **City pilot:** real cameras, real helplines
- 5 **New sectors:** hospitals (fall detection), schools, factories

Cameras are already there. VisionGuard gives them a brain.

Detect · Decide · Dispatch

Thank you — questions?